Theorem. A function's type is usually read as a constraint on its values. In a data-oriented machine it is something stronger: a description of the *geometry* of the data being walked — and therefore of the control flow itself. Where the data branches, the code parallelizes; where it merges, the code synchronizes. The signature carries both.

The coefficient is the geometry

Multiplicity lives in the type as a *coefficient* — a (min, max) pair on every type. Four shapes do the work:

coefficient	meaning	form
$\{0\}$	nothing	seed
$\{?\}$	zero or one	maybe / filter
$\{1\}$	exactly one	map
$\{*\}$	any number	gather / scatter

An instruction is a morphism $f : A^{c_{\text{dom}}} \to B^{c_{\text{rng}}}$, and its *form* is read off the coefficient pair:

form	signature	what it is
map	$\{1\} \to \{1\}$	one in, one out — pointwise, freely parallel
scatter	$\{1\} \to \{*\}$	one in, many out — a *branch*
reduce	$\{*\} \to \{1\}$	many in, one out — a *merge*
seed	$\{0\} \to A$	from nothing

Live proof. A coefficient is not annotation; it is arithmetic. `count()` sums the coefficients, and `sum()` lets a merge account for them:

```
mtron> {1,2,3}.count()              [-- 3: sum of coefficients --]
==>3
mtron> {1,2,{10}3}.count()          [-- 12: coefficients add (1+1+10) --]
==>12
mtron> {int{2}::1,int{3}::2}.sum()  [-- int::8: the merge accounts for the coefficient --]
==>8
```

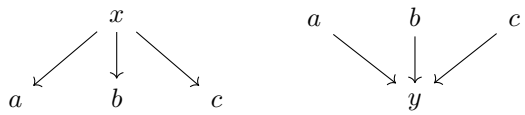
The tree walk is the schedule

Data lives at addresses, and addresses are paths in a tree. Descending a node with N children *splits* the traversal — the multiplicity grows to N. Ascending *merges* it, and the coefficient is the bulk that carries the count back up.

Live proof. `>>` is `rshift` — one level deeper into the tree. Read the running machine's own thread tree: each `>>` is a scatter that fans a node out into its children, and each `.count()` is the gather that folds them back to a single number:

```
mtron> /sys/thread.>>
==>/sys/thread/executor
==>/sys/thread/main
mtron> /sys/thread.>>.count()
==>2
mtron> /sys/thread.>>.>>.count()
==>7
```

The branching factor *is* the coefficient. Descend and the count grows; gather and it folds back to one number. The two directions are the two fundamental operations:



scatter $\{1\} \to \{*\}$ (left) and gather $\{*\} \to \{1\}$ (right)

Descending *is* the fan-out; ascending *is* the fan-in. The data structure hands the compiler its own shape.

Category theory, in one line

Scatter is a *cocartesian fan-out* (one wire, many); gather is the *limit* that reunifies them. The coefficient is a *measure on the tree* — the multiplicity of the fiber over a point — and an instruction's form is how that measure changes across the morphism.

Corollary. The compiler doesn't infer parallelism from control flow; it reads it off the data's geometry. Branch $\Rightarrow$ parallelize. Merge $\Rightarrow$ barrier. Coefficient $\Rightarrow$ signature. No annotations, no static control-flow analysis — the data carries the plan.